

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНЫМ РАБОТАМ № 5

**«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Кретов Иван Алексеевич

Факультет прикладной информатики

Группа K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватели Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.
Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

Указание. Работа выполняется в консоли SQL Shell (psql).

Выполнение:

Процедуры:

1. Поиск билетов в заданный пункт назначения

```
CREATE OR REPLACE FUNCTION schema.search_tickets(p_arrival_airport_id UUID)
RETURNS TABLE(
    id_flight UUID,
    dep_date DATE,
    departure_airport VARCHAR(3),
    arrival_airport VARCHAR(3),
    plane_type VARCHAR(20),
    free_seats INTEGER
)
LANGUAGE SQL
AS $$
SELECT
    f.id_flight,
    r.departure_time AS dep_date,
    a_dep.code_name AS departure_airport,
    a_arr.code_name AS arrival_airport,
    p.type AS plane_type,
    (SELECT COUNT(*) FROM schema.seat s
     WHERE s.id_flight = f.id_flight AND s.status = 'Свободно') AS free_seats
FROM schema.flight f
JOIN schema.route r ON f.id_route = r.id_route
JOIN schema.airport a_dep ON r.id_departure_airport = a_dep.id_airport
JOIN schema.airport a_arr ON r.id_arrival_airport = a_arr.id_airport
JOIN schema.plane p ON f.id_plane = p.id_plane
LEFT JOIN schema.seat s ON f.id_flight = s.id_flight
LEFT JOIN schema.seat_type st ON s.id_seat_type = st.id_seat_type
WHERE a_arr.id_airport = p_arrival_airport_id AND f.status <> 'Отменён'
GROUP BY f.id_flight, r.departure_time, a_dep.code_name, a_arr.code_name, p.type;
$$;
```

```
ivan — psql - runpsql.sh — 133x28

airport_lab3=#
airport_lab3=# SELECT * FROM schema.search_tickets('4adbb72-540c-4fcb-a561-8a4182189343');
 id_flight | dep_date | departure_airport | arrival_airport | plane_type | free_seats
-----|-----|-----|-----|-----|-----
 2b858873-6c23-43f0-b317-6e2a44a75a27 | 2026-03-17 | AAC | AAD | Sukhoi Superjet 100 | 60
 94bb43ef-8dbe-4a24-a0fe-2c65edc5feab | 2026-04-28 | TTT | AAD | Boeing 737 | 150
(2 rows)

airport_lab3=# SELECT * FROM schema.search_tickets('ea63df55-9dbf-4e1c-a88b-b3d52fa9b2c6');
 id_flight | dep_date | departure_airport | arrival_airport | plane_type | free_seats
-----|-----|-----|-----|-----|-----
 075eec0a-7d7e-4543-9683-a6bddce81ed6 | 2026-04-30 | TTT | TTT | Boeing 737 | 150
(1 row)

airport_lab3=# SELECT * FROM schema.search_tickets('4665fe02-292a-4561-b283-c97cf7261903');
 id_flight | dep_date | departure_airport | arrival_airport | plane_type | free_seats
-----|-----|-----|-----|-----|-----
 36f07d84-334a-4e25-9217-a5a17bba139a | 2026-04-24 | TTT | NMA | Boeing 737 | 150
(1 row)

airport_lab3=# SELECT * FROM schema.search_tickets('ff0ba6e1-4522-43f8-a94a-2c2a165b141f');
 id_flight | dep_date | departure_airport | arrival_airport | plane_type | free_seats
-----|-----|-----|-----|-----|-----
 6eeaff72-b210-446b-87e0-350423925eb1 | 2026-05-10 | TTT | AAP | Boeing 737 | 150
 8e623c80-2aab-44e1-8b3a-4dd9b78b841f | 2026-03-29 | AAO | AAP | Airbus A380 | 300
(2 rows)

airport_lab3=#
```

2. Создание новой кассы продажи билетов

```
CREATE OR REPLACE PROCEDURE schema.create_cashbox(
    IN p_status VARCHAR(10),
    IN p_address VARCHAR(50)
)
LANGUAGE plpgsql
AS $$
DECLARE
    new_id UUID;
BEGIN
    new_id := gen_random_uuid();
    INSERT INTO schema.cashbox (id_cashbox, status, address)
    VALUES (new_id, p_status, p_address);
END;
$$;
```

```
[airport_lab3=# SELECT * FROM schema.cashbox
[airport_lab3=# ;
      id_cashbox          | status | address
-----+-----+-----
 b5a8e722-7b66-4ffc-b143-249514e67383 | Работает | г. Москва, ул. Пушкина, д. 13
(1 row)

[airport_lab3=# CALL schema.create_cashbox('Работает','г. Москва, ул. Тверская, д. 1');
CALL
[airport_lab3=# SELECT * FROM schema.cashbox;
      id_cashbox          | status | address
-----+-----+-----
 b5a8e722-7b66-4ffc-b143-249514e67383 | Работает | г. Москва, ул. Пушкина, д. 13
 5d132fc7-ea3c-4bd4-b33b-9646cf81da00 | Работает | г. Москва, ул. Тверская, д. 1
(2 rows)

airport_lab3=#
```

3. Определить расход топлива по всем маршрутам за истекший месяц

```
CREATE OR REPLACE FUNCTION schema.fuel_consumption_last_month()
RETURNS TABLE(
    route_id UUID,
    departure_code VARCHAR(3),
    arrival_code VARCHAR(3),
    total_fuel_liters NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    start_date DATE := CURRENT_DATE - INTERVAL '1 month';
    end_date DATE := CURRENT_DATE;
BEGIN
    RETURN QUERY
    SELECT
        r.id_route,
        a_dep.code_name,
        a_arr.code_name,
        ROUND(SUM(EXTRACT(EPOCH FROM f.actual_arrival - f.actual_departure) / 3600 *
p.fuel_consumption), 2) AS total_fuel
    FROM schema.flight f
    JOIN schema.route r ON f.id_route = r.id_route
```

```

JOIN schema.plane p ON f.id_plane = p.id_plane
JOIN schema.airport a_dep ON r.id_departure_airport = a_dep.id_airport
JOIN schema.airport a_arr ON r.id_arrival_airport = a_arr.id_airport
WHERE f.actual_departure BETWEEN start_date AND end_date
      AND f.status = 'Выполнен'
GROUP BY r.id_route, a_dep.code_name, a_arr.code_name;
END;
$$;

```

```

[airport_lab3=# SELECT * FROM schema.fuel_consumption_last_month();
               route_id | departure_code | arrival_code | total_fuel_liters
-----+-----+-----+-----
1b5ecf4c-81a5-4ab6-85df-140ef7f08768 | ABW            | ABX            | 35000.00
22e29b23-e7d0-4729-a232-06c686891918 | ACE            | ACF            | 5400.00
81301436-9dc9-4213-aea7-21f98f75888b | ABS            | ABT            | 9000.00
e68e459a-bfcb-46ce-a2f9-289464c7a640 | ACA            | ACB            | 18000.00
(4 rows)

airport_lab3=# █

```

Триггеры:

1. Освобождения места после удаления (возврата) билета

```
CREATE OR REPLACE FUNCTION schema.trg_seat_free()
RETURNS trigger AS $$
BEGIN
    UPDATE schema.seat SET status = 'Свободно'
    WHERE id_seat = OLD.id_seat;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ticket_after_delete
AFTER DELETE ON schema.ticket
FOR EACH ROW EXECUTE FUNCTION schema.trg_seat_free();
```

```
airport_lab3=# select * from schema.seat where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
 id_seat | id_flight | id_seat_type | status
-----+-----+-----+-----
 d6d9061c-3a2e-4888-81f0-a9c2d5dbaffc | 85a5d3f0-47dc-4143-983b-9709cc26b5e3 | b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c | Занято
(1 row)

airport_lab3=# delete from schema.ticket where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
DELETE 1
airport_lab3=# select * from schema.seat where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
 id_seat | id_flight | id_seat_type | status
-----+-----+-----+-----
 d6d9061c-3a2e-4888-81f0-a9c2d5dbaffc | 85a5d3f0-47dc-4143-983b-9709cc26b5e3 | b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c | Свободно
(1 row)

airport_lab3=#
```

2. Занятие места после покупки билета

```
CREATE OR REPLACE FUNCTION schema.trg_seat_occupy()
RETURNS trigger AS $$
BEGIN
    UPDATE schema.seat
    SET status = 'Занято'
    WHERE id_seat = NEW.id_seat;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ticket_after_insert
AFTER INSERT ON schema.ticket
FOR EACH ROW EXECUTE FUNCTION schema.trg_seat_occupy();
```

```
airport_lab3=# select * from schema.ticket where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
 id_ticket | id_cashbox | id_passenger_passport | id_seat | id_seat_type | total_price | payment_date | payment_status
-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

airport_lab3=# select * from schema.seat where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
 id_seat | id_flight | id_seat_type | status
-----+-----+-----+-----
 d6d9061c-3a2e-4888-81f0-a9c2d5dbaffc | 85a5d3f0-47dc-4143-983b-9709cc26b5e3 | b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c | Свободно
(1 row)

airport_lab3=# insert into schema.ticket (id_ticket, id_cashbox, id_passenger_passport, id_seat, id_seat_type, total_price, payment_date, payment_status) values (gen_random_uuid(), 'b5a8e722-7b66-4ffc-b143-249514e67383', '02c6781b-1d67-4290-a323-71c9cb9fc29b', 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc', 'b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c', 5000, CURRENT_DATE, 'Оплачен');
INSERT 0 1
airport_lab3=# select * from schema.seat where id_seat = 'd6d9061c-3a2e-4888-81f0-a9c2d5dbaffc';
 id_seat | id_flight | id_seat_type | status
-----+-----+-----+-----
 d6d9061c-3a2e-4888-81f0-a9c2d5dbaffc | 85a5d3f0-47dc-4143-983b-9709cc26b5e3 | b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c | Занято
(1 row)

airport_lab3=#
```

3. Запрет на покупку уже занятого места

```
CREATE OR REPLACE FUNCTION schema.trg_check_seat_free()
RETURNS trigger AS $$
DECLARE
    seat_status VARCHAR(20);
BEGIN
    SELECT status INTO seat_status FROM schema.seat
    WHERE id_seat = NEW.id_seat;

    IF seat_status <> 'Свободно' THEN
        RAISE EXCEPTION 'Данное место уже занято (id_seat = %)', NEW.id_seat;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_ticket_before_insert
BEFORE INSERT ON schema.ticket
FOR EACH ROW EXECUTE FUNCTION schema.trg_check_seat_free();
```

```
airport_lab3=# select * from schema.ticket where id_seat = 'd6d9861c-3a2e-4888-81f8-a9c2d5dbaffc';
 id_ticket | id_cashbox | id_passenger_passport | id_seat | id_seat_type | total_price | payment_date | payment_status
(1 row)
 e51ae546-2bef-46d2-84d3-922821837de5 | b5a8e722-7b66-4ffc-b143-249514e67383 | 82c6781b-1d67-4298-a323-71c9cb9fc29b | d6d9861c-3a2e-4888-81f8-a9c2d5dbaffc | b07e3f6a-85cc-4c6a-87d5-7d32d5d86f1c | 5000.00 | 2026-05-25 | Отменен

airport_lab3=# insert into schema.ticket (id_ticket, id_cashbox, id_passenger_passport, id_seat, id_seat_type, total_price, payment_date, payment_status) values (gen_random_uuid(), 'b5a8e722-7b66-4ffc-b143-249514e67383', '82c6781b-1d67-4298-a323-71c9cb9fc29b', 'd6d9861c-3a2e-4888-81f8-a9c2d5dbaffc', 'b07e3f6a-85cc-4c6a-87d5-7d32d5d86f1c', 5000, CURRENT_DATE, 'Отменен');
ERROR: Данное место уже занято (id_seat = d6d9861c-3a2e-4888-81f8-a9c2d5dbaffc)
CONTEXT: PL/pgSQL function schema.trg_check_seat_free() line 9 at 8:150
airport_lab3=# select * from schema.ticket where id_seat = 'd6d9861c-3a2e-4888-81f8-a9c2d5dbaffc';
 id_ticket | id_cashbox | id_passenger_passport | id_seat | id_seat_type | total_price | payment_date | payment_status
(1 row)
 e51ae546-2bef-46d2-84d3-922821837de5 | b5a8e722-7b66-4ffc-b143-249514e67383 | 82c6781b-1d67-4298-a323-71c9cb9fc29b | d6d9861c-3a2e-4888-81f8-a9c2d5dbaffc | b07e3f6a-85cc-4c6a-87d5-7d32d5d86f1c | 5000.00 | 2026-05-25 | Отменен

airport_lab3=#
```

4. Проверка доступа сотрудника по мед осмотру

```
CREATE OR REPLACE FUNCTION schema.trg_crew_medical()
RETURNS trigger AS $$
BEGIN
    IF NEW.medical_status <> 'Допущен' THEN
        RAISE EXCEPTION 'Член экипажа не допущен по медосмотру (статус: %)',
NEW.medical_status;
    END IF;
    IF NEW.rejection_reason IS NOT NULL THEN
        RAISE EXCEPTION 'Есть причина недопуска: %', NEW.rejection_reason;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_flight_crew_medical
BEFORE INSERT OR UPDATE ON schema.flight_crew
FOR EACH ROW EXECUTE FUNCTION schema.trg_crew_medical();
```

```
airport_lab3=# insert into schema.flight_crew (id_flight_crew, id_flight, id_employee_passport, medical_status, medical_check_date) value
s (gen_random_uuid(), '580da643-22b1-46d3-a4cc-95fe83f9bab2', '0f9ef58f-d55b-4e39-8ef3-9b477b64ded8', 'Не допущен', CURRENT_DATE);
ERROR: Член экипажа не допущен по медосмотру (статус: Не допущен)
CONTEXT: PL/pgSQL function schema.trg_crew_medical() line 4 at RAISE
airport_lab3=# insert into schema.flight_crew (id_flight_crew, id_flight, id_employee_passport, medical_status, medical_check_date) value
s (gen_random_uuid(), '580da643-22b1-46d3-a4cc-95fe83f9bab2', '0f9ef58f-d55b-4e39-8ef3-9b477b64ded8', 'Допущен', CURRENT_DATE);
INSERT 0 1
airport_lab3=#
```

5. Проверка даты оплаты

```
CREATE OR REPLACE FUNCTION schema.trg_ticket_payment()
RETURNS trigger AS $$
BEGIN
    IF NEW.payment_status = 'Оплачен' THEN
        IF NEW.payment_date IS NULL THEN
            RAISE EXCEPTION 'Для оплаченного билета должна быть указана дата платежа';
        END IF;
        IF NEW.payment_date > CURRENT_DATE THEN
            RAISE EXCEPTION 'Дата оплаты не может быть в будущем';
        END IF;
    ELSIF NEW.payment_status = 'Не оплачен' THEN
        IF NEW.payment_date IS NOT NULL THEN
            RAISE EXCEPTION 'У неоплаченного билета не должно быть даты платежа';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_ticket_payment_consistency
BEFORE INSERT OR UPDATE ON schema.ticket
FOR EACH ROW EXECUTE FUNCTION schema.trg_ticket_payment();
```

```
airport_lab3=# insert into schema.ticket (id_ticket, id_cashbox, id_passenger_passport, id_seat, id_seat_type, total_price, payment_date,
payment_status) values (gen_random_uuid(), 'b5a8e722-7b66-4ffc-b143-249514e67383', '02c6781b-1d67-4290-a323-71c9cb9fc29b', 'd6d9061c-3a2
e-40888-81f0-a9c2d5dbaffc', 'b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c', 5000, CURRENT_DATE, 'Не оплачен');
ERROR:  У неоплаченного билета не должно быть даты платежа
CONTEXT:  PL/pgSQL function schema.trg_ticket_payment() line 12 at RAISE
airport_lab3=# insert into schema.ticket (id_ticket, id_cashbox, id_passenger_passport, id_seat, id_seat_type, total_price, payment_date,
payment_status) values (gen_random_uuid(), 'b5a8e722-7b66-4ffc-b143-249514e67383', '02c6781b-1d67-4290-a323-71c9cb9fc29b', 'd6d9061c-3a2
e-40888-81f0-a9c2d5dbaffc', 'b07e3f6a-e5cc-4c6a-87d5-7d32d5d06f1c', 5000, CURRENT_DATE, 'Оплачен');
INSERT 0 1
airport_lab3=#
```

6. Автоматическое создание мест после добавления нового рейса

```
CREATE OR REPLACE FUNCTION schema.trg_generate_seats_for_flight()
RETURNS trigger AS $$
DECLARE
    plane_capacity INTEGER;
    seat_type_id UUID;
    i INTEGER;
BEGIN
    SELECT seat_count INTO plane_capacity
    FROM schema.plane
    WHERE id_plane = NEW.id_plane;

    SELECT id_seat_type INTO seat_type_id
    FROM schema.seat_type
    WHERE id_route = NEW.id_route
    LIMIT 1;

    FOR i IN 1..plane_capacity LOOP
        INSERT INTO schema.seat (id_seat, id_flight, id_seat_type, status)
        VALUES (gen_random_uuid(), NEW.id_flight, seat_type_id, 'free');
    END LOOP;

    RETURN NEW;
END;
```



```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_flight_after_insert
AFTER INSERT ON schema.flight
FOR EACH ROW EXECUTE FUNCTION schema.trg_generate_seats_for_flight();
```

```
[airport_lab3=# select COUNT(*) from schema.seat;
count
-----
85900
(1 row)

[airport_lab3=# insert into schema.flight(id_flight, id_route, id_plane, planned_departure, planned_arrival, status) values (gen_random_uuid(), 'a495fd6b-10f7-47fd-aa5d-2dbe5cfbfc06', '93d1d975-f7fc-4347-843a-247b88568b05', '2026-06-01 12:00:00', '2026-06-01 16:00:00', 'По расписанию');
INSERT 0 1
[airport_lab3=# select COUNT(*) from schema.seat;
count
-----
86080
(1 row)

airport_lab3=#
```

7. Автоматическое добавление налета самолету после завершения рейса

```
CREATE OR REPLACE FUNCTION schema.trg_update_plane_hours_on_completion()
RETURNS trigger AS $$
```

```
DECLARE
```

```
    flight_duration_hours NUMERIC;
```

```
BEGIN
```

```
    IF NEW.status = 'Выполнен' THEN
```

```
        flight_duration_hours := EXTRACT(EPOCH FROM (NEW.actual_arrival -
NEW.actual_departure)) / 3600.0;
```

```
        UPDATE schema.plane
```

```
        SET flight_hours = flight_hours + flight_duration_hours::INT
```

```
        WHERE id_plane = NEW.id_plane;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_flight_after_update_completed
```

```
AFTER UPDATE OF status ON schema.flight
```

```
FOR EACH ROW EXECUTE FUNCTION schema.trg_update_plane_hours_on_completion();
```

```
[airport_lab3=# select flight_hours from schema.plane where id_plane = '93d1d975-f7fc-4347-843a-247b88568b05';
flight_hours
-----
200
(1 row)

[airport_lab3=# update schema.flight set status = 'Выполнен' where id_flight = '85a5d3f0-47dc-4143-983b-9709cc26b5e3';
UPDATE 1
[airport_lab3=# select flight_hours from schema.plane where id_plane = '93d1d975-f7fc-4347-843a-247b88568b05';
flight_hours
-----
202
(1 row)

airport_lab3=#
```

Выводы:

В ходе выполнения лабораторной работы были разработаны и внедрены хранимые функции и процедуры, реализующие операции предметной области аэропорта. Все процедурные объекты успешно созданы в схеме базы данных, протестированы и корректно возвращают ожидаемые результаты. Для обеспечения автоматического поддержания целостности и непротиворечивости данных было создано семь триггеров согласно варианту 2.1. Триггеры охватывают основные процессы предметной области и реагируют на события вставки, удаления и обновления данных. Все триггеры были протестированы отдельно с использованием как корректных, так и заведомо ошибочных данных, что подтвердило их работоспособность и правильную реакцию на нарушения ограничений. Использование триггеров позволило переложить часть бизнес-логики с прикладного уровня на уровень СУБД, снизив риск появления несогласованных данных и упростив разработку клиентских приложений. В процессе работы были изучены особенности создания триггерных функций, работа с переменными OLD и NEW. Практическое задание выполнено в полном объёме, полученные навыки могут быть применены для разработки более сложных систем обработки данных в PostgreSQL